

April 6th, 2026



Reliable Rentals

Logical Database Design | Part 2

DSC623 – Database Systems for Data Science | Spring 2026

Prof. Vanessa Aguiar

Christian Hernandez • Gabriella Escobar Cukier • Anastasiya Drandarov

A. Relational Schema

The following relations were derived from the conceptual model. Primary keys are underlined; foreign keys are marked with an asterisk (*). Alternate keys are noted where applicable.

Outlet (outlet_number, address [AK], phone_number, fax_number)

Vehicle (registration_number, model, make, engine_size, capacity, current_mileage, daily_hire_rate, outlet_number*)

Client (client_number, first_name, last_name, home_address, phone_number, date_of_birth, driving_license_number [AK])

HireAgreement (hire_number, start_date, expected_end_date, mileage_before, mileage_after, client_number*, registration_number*, staff_number*)

Staff (staff_number, first_name, last_name, home_address, home_phone_number, date_of_birth, sex, date_joined_company, job_title, salary, outlet_number*)

* = Foreign Key [AK] = Alternate Key = Primary Key

Assumption: staff_number is included as FK in HireAgreement under the assumption that every hire agreement is processed by a staff member, and the company would like to store that information (**this was discussed with the professor and it is ok**). The case study does not state this explicitly; this assumption is flagged for instructor confirmation.

B. Normalization to 3NF

The normalization process begins from a single universal relation containing all attributes from the case study and proceeds through 1NF, 2NF, and 3NF.

Original Universal Relation

All attributes combined into one flat table. The primary key is hire_number, since knowing the hire number uniquely identifies a row: it determines the registration_number (one vehicle per agreement), the client_number, and the staff_number, which in turn identifies the outlet_number.

```
UniversalRelation (hire_number, start_date, expected_end_date,  
    mileage_before, mileage_after, registration_number, model, make,  
    engine_size, capacity, current_mileage, daily_hire_rate,  
    outlet_number, address, phone_number, fax_number,  
    client_number, first_name, last_name, home_address,  
    date_of_birth, driving_license_number,  
    staff_number, home_phone_number, sex,  
    date_joined_company, job_title, salary)
```

Functional Dependencies Identified

- hire_number → start_date, expected_end_date, mileage_before, mileage_after, registration_number, client_number, staff_number
- registration_number → model, make, engine_size, capacity, current_mileage, daily_hire_rate — transitive
- outlet_number → address, phone_number, fax_number — transitive
- client_number → first_name, last_name, home_address, phone_number, date_of_birth, driving_license_number — transitive
- staff_number → first_name, last_name, home_address, home_phone_number, date_of_birth, sex, date_joined_company, job_title, salary, outlet_number — transitive

1NF

The original table is already in 1NF. All attributes are atomic (composite name is split into first_name and last_name), and there are no repeating groups.

2NF

2NF requires removing partial dependencies. Since hire_number alone is the primary key and is not composite, there are no partial dependencies. The table is already in 2NF.

3NF

3NF requires removing transitive dependencies. All five identified above are resolved by extracting the dependent attributes into their own tables:

```
HireAgreement (hire_number (PK), start_date, expected_end_date,  
              mileage_before, mileage_after,  
              registration_number (FK), client_number (FK),  
              staff_number (FK))
```

```
Vehicle (registration_number (PK), model, make, engine_size,  
        capacity, current_mileage, daily_hire_rate)
```

```
Client (client_number (PK), first_name, last_name, home_address,  
       phone_number, date_of_birth, driving_license_number)
```

```
Staff (staff_number (PK), first_name, last_name, home_address,  
      home_phone_number, date_of_birth, sex,  
      date_joined_company, job_title, salary, outlet_number (FK))
```

```
Outlet (outlet_number (PK), address, phone_number, fax_number)
```

All five tables are now in 3NF: every non-key attribute depends on the primary key, the whole key, and nothing but the key.

C. User Transactions

The logical model was validated against five user transactions. The majority involve more than one entity. No SQL code is used; each transaction describes the navigation path through the relationships.

Transaction 1 – Vehicles at a specific outlet and their hire rate

Query: Which vehicles are currently stored at Outlet 3, and what is their daily hire rate?

1. Go to the Outlet table, filter where outlet_number = 3 to confirm the outlet exists.
2. Go to the Vehicle table, filter where outlet_number = 3 (FK) to get all vehicles at that outlet.
3. From those rows, return: registration_number, model, make, daily_hire_rate.

Entities traversed: Outlet → Vehicle.

Transaction 2 – Hire history for a specific client

Query: Show all hire agreements signed by client 101, including the vehicle rented and the hire dates.

1. Go to the Client table, filter where client_number = 101 to get first_name and last_name.
2. Go to the HireAgreement table, filter where client_number = 101 (FK) to get all agreements for that client.
3. For each agreement, take registration_number (FK) and look up the Vehicle table to get model and make.
4. Return: hire_number, start_date, expected_end_date, model, make, mileage_before, mileage_after.

Entities traversed: Client → HireAgreement → Vehicle.

Transaction 3 – All ongoing hires with client and outlet info

Query: List all ongoing hires (not yet returned), with the client's name and the outlet where the vehicle is based.

1. Go to the HireAgreement table, filter where mileage_after is null to get only active hires.
2. For each row, take client_number (FK) and look up the Client table to get first_name and last_name.
3. Take registration_number (FK) and look up the Vehicle table to get outlet_number.
4. Take that outlet_number (FK) and look up the Outlet table to get the address.
5. Return: hire_number, client name, start_date, registration_number, outlet address.

Entities traversed: HireAgreement → Client, HireAgreement → Vehicle → Outlet.

Transaction 4 – Clients who rented vehicles from a specific outlet

Query: List all clients who have ever rented a vehicle belonging to Outlet 5, along with the vehicle they rented and the hire dates.

1. Go to the Vehicle table, filter where outlet_number = 5 (FK) to get all registration_numbers based at that outlet.
2. Go to the HireAgreement table, filter where registration_number matches any of those vehicles (FK).
3. For each agreement, take client_number (FK) and look up the Client table to get first_name, last_name, and phone_number.
4. Take registration_number (FK) and look up the Vehicle table to get model and make.
5. Return: client name, phone_number, model, make, start_date, expected_end_date, mileage_before, mileage_after.

Entities traversed: Outlet → Vehicle → HireAgreement → Client.

Transaction 5 – Total charge for a hire agreement

Query: What is the total charge for hire agreement 20, based on the number of days and the vehicle's daily hire rate?

1. Go to the HireAgreement table, filter where hire_number = 20 to get start_date, expected_end_date, mileage_before, mileage_after, and registration_number.
2. Calculate number of days hired: $\text{expected_end_date} - \text{start_date} + 1$.
3. Take registration_number (FK) and look up the Vehicle table to get daily_hire_rate.
4. Calculate total cost: $\text{number of days} \times \text{daily_hire_rate}$.
5. Calculate distance driven: $\text{mileage_after} - \text{mileage_before}$ (only if mileage_after is not null).
6. Return: hire_number, total cost, distance driven.

Entities traversed: HireAgreement → Vehicle.

D. Integrity Constraints

D.I – Primary Key Constraints

Each entity has exactly one primary key. All primary keys must be unique and NOT NULL.

Entity	Primary Key
Outlet	outlet_number
Vehicle	registration_number
Client	client_number
Hire Agreement	hire_number
Staff	staff_number

D.II – Referential Integrity / Foreign Key Constraints

All foreign keys must have valid, non-null values that exist in the referenced parent table.

Foreign Key (Child Table)	References (Parent Table)	On Delete
Vehicle.outlet_number	Outlet.outlet_number	RESTRICT
HireAgreement.client_number	Client.client_number	RESTRICT
HireAgreement.registration_number	Vehicle.registration_number	RESTRICT
HireAgreement.staff_number	Staff.staff_number	RESTRICT
Staff.outlet_number	Outlet.outlet_number	RESTRICT

Deletion is restricted on all parent records with active child references. An Outlet cannot be deleted while Staff or Vehicles are linked to it. A Vehicle or Client cannot be deleted while HireAgreements reference them.

D.III – Alternate Key Constraints

- Client.driving_license_number – must be unique and non-null; no two clients may share a driving license number.
- Outlet.address – assumed unique; no two outlets operate from the same physical address.

D.IV – Required Data

The following attributes cannot be NULL:

Entity	Required (NOT NULL) Attributes
Outlet	outlet_number, address, phone_number
Vehicle	registration_number, model, make, daily_hire_rate, outlet_number
Client	client_number, first_name, last_name, phone_number, date_of_birth, driving_license_number
Hire Agreement	hire_number, start_date, client_number, registration_number, staff_number
Staff	staff_number, first_name, last_name, job_title, salary, outlet_number, date_joined_company

D.V – Attribute Domain Constraints

- salary: numeric, must be > 0 .
- daily_hire_rate: numeric, must be > 0 .
- current_mileage, mileage_before, mileage_after: numeric, must be ≥ 0 .
- mileage_after $\geq$ mileage_before within any single hire agreement.
- expected_end_date $\geq$ start_date within any hire agreement.
- date_of_birth: valid calendar date in the past; client must be at least 18 years of age.
- sex: restricted to a defined set of values (e.g., 'M', 'F', 'Other').
- engine_size, capacity: numeric, must be > 0 .

D.VI – General Constraints

- A vehicle may not appear in two overlapping active hire agreements simultaneously. Date ranges for the same registration_number must not overlap.
- Each staff member is associated with exactly one outlet at any given time, enforced via the single outlet_number FK on the Staff table.
- HireAgreement.mileage_before must equal Vehicle.current_mileage at the moment the agreement is created, ensuring consistency between the vehicle record and the hire record.
- When mileage_after is recorded upon return, Vehicle.current_mileage must be updated to that same value, keeping the vehicle's mileage current.

E. E-R Diagram – Logical Level

The logical E-R diagram below was built in PowerPoint using shapes, following the same visual style as Part 1. Unlike the Part 1 conceptual diagram, this logical diagram includes foreign keys as explicit attributes within each entity box — as required by the assignment. Primary keys are underlined; foreign keys are shown in *italic*. Relationship cardinalities use the notation 1..1, 1..*, and 0..* consistent with Part 1.

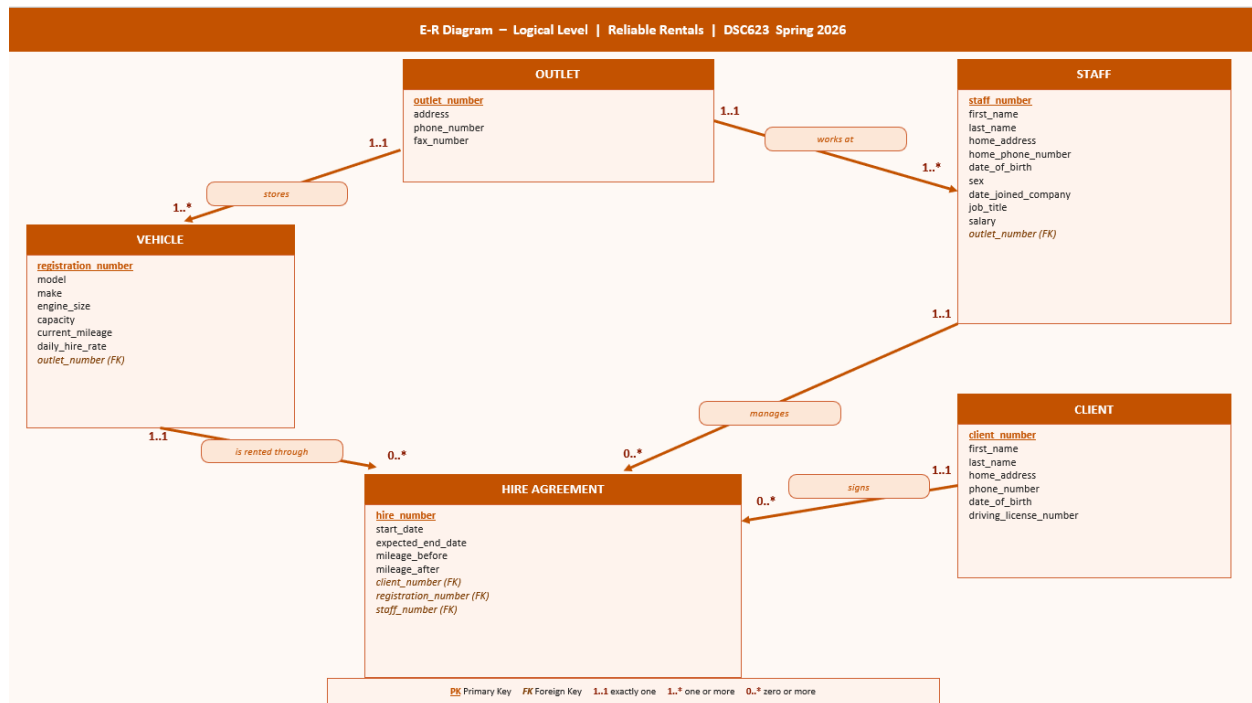


Figure 1. Logical E-R Diagram – Reliable Rentals

Multiplicity constraints represented in the diagram:

Entity	Multiplicity	Relationship	Multiplicity	Entity
Outlet	1..1	stores	1..*	Vehicle
Vehicle	1..1	is rented through	0..*	Hire Agreement
Client	1..1	signs	0..*	Hire Agreement
Staff	1..*	works at	1..1	Outlet
Staff	1..1	manages	0..*	Hire Agreement